Form & Repository Functions — API Reference

The V6 API endpoints used for:

1. **Form file upload with OCR** — `uploadForOcr` → `uploadWithOcr`
2. **Forms** — create / update / delete a form
3. **Form entries** — create / update / delete / read entries
4. **Repository file upload**
5. **Repository structure & items** — folder tree and file list
6. **Indexed vs archived files** — which list shows which files

All endpoints need `Authorization: Bearer <accessToken>` and a tenant user (policy `TenantUser`). The tenant comes from the token. Errors return `{ "error": "..." }` with `400` unless noted.

Quick map

#	Function	Method & route	Controller
1a	OCR only (no save)	`POST /api/uploadAndIndex/uploadForOcr`	`UploadAndIndexController`
1b	Stage file with OCR metadata	`POST /api/uploadAndIndex/uploadWithOcr`	`UploadAndIndexController`
1c	Save fields + archive staged file	`PUT /api/uploadAndIndex/index/{fileId}`	`UploadAndIndexController`
1d	Update staged fields only	`PUT /api/uploadAndIndex/index/{fileId}/fields`	`UploadAndIndexController`
1e	Delete staged files	`POST /api/uploadAndIndex/index/deleteFiles`	`UploadAndIndexController`
2a	Create form	`POST /api/form`	`FormController`
2b	Update form	`PUT /api/form/{formId}`	`FormController`
2c	Delete form (soft)	`DELETE /api/form/{formId}`	`FormController`
2d	Get form (with formJson)	`GET /api/form/{formId}`	`FormController`
2e	List forms	`GET /api/form/all`, `POST /api/form/all`	`FormController`
3a	Create entry	`POST /api/form/{formId}/entry/00000000-0000-0000-0000-000000000000`	`FormController`
3b	Update entry	`PUT /api/form/{formId}/entry/{entryId}`	`FormController`
3c	Upsert entry by column	`POST /api/form/{formId}/entry/upsert`	`FormController`
3d	Update entry by column	`PUT /api/form/{formId}/entry/by-column`	`FormController`
3e	Get entry	`GET /api/form/{formId}/entry/{entryId}`	`FormController`
3h	Delete entry (soft)	`DELETE /api/form/{formId}/entry/{entryId}`	`FormController`
3f	List entries	`POST /api/form/{formId}/entry/all`	`FormController`
3g	Form controls (field schema)	`GET /api/form/{formId}/entry/controls`	`FormController`

#	Function	Method & route	Controller
4a	Upload file (folder layout)	`POST /api/repositories/{repoId}/items/upload-archive`	`RepositoriesController`
4b	Upload file (flat path)	`POST /api/repositories/{repoId}/items/upload`	`RepositoriesController`
4c	Update item metadata	`PATCH /api/repositories/{repoId}/items/{itemId}/metadata`	`RepositoriesController`
4d	Delete item (soft)	`DELETE /api/repositories/{repoId}/items/{itemId}`	`RepositoriesController`
4e	Metadata edit history	`GET /api/repositories/{repoId}/items/{itemId}/metadata/history`	`RepositoriesController`
5a	Folder structure	`GET /api/repositories/{repoId}/browse/structure`	`RepositoriesController`
5b	Next folder level	`GET /api/repositories/{repoId}/browse/children`	`RepositoriesController`
5c	Item list (paged)	`GET /api/repositories/{repoId}/items`	`RepositoriesController`
5d	Item list (filters in body)	`POST /api/repositories/{repoId}/items/query`	`RepositoriesController`
5e	Filter fields	`GET /api/repositories/{repoId}/items/filter-fields`	`RepositoriesController`
5f	Item detail / file	`GET .../items/{itemId}`, `GET .../items/{itemId}/file`	`RepositoriesController`
6a	Files waiting to be indexed	`POST /api/uploadAndIndex/index/all`	`UploadAndIndexController`
6c	Bulk upload progress	`GET /api/uploadAndIndex/bulkUpload/jobs/{jobId}`	`UploadAndIndexController`

---

1. Form file upload with OCR

Typical flow on a form with a file/receipt control:

1. User picks a file → **`uploadForOcr`** → show extracted values in the form (nothing saved yet).
2. User reviews/edits → **`uploadWithOcr`** → file is staged, returns `fileId` + `filePath`.
3. Save the form entry / start the ticket with the `fileId` / `filePath`.
4. Later, archive the staged file into the repository with **`PUT index/{fileId}`**.

1a. `POST /api/uploadAndIndex/uploadForOcr`

Sends the file to the Python OCR service and returns the result. **Does not save** the file.

`multipart/form-data` (max 100 MB):

Field	Required	Notes
`file`	**Yes**	The document.
`repositoryId`	**Yes**	Repository GUID. Its fields are the default OCR fields.
`fields`	No	Override which fields to extract. JSON array — either `["Invoice Number,SHORT_TEXT","Invoice Date,DATE"]` or `[{"name":"Invoice Number","type":"SHORT_TEXT"}]`. Types: `SHORT_TEXT`, `LONG_TEXT`, `NUMBER`, `DATE`, `TABLE`. Omit to use the repository's fields.
`pageNo`	No	Page(s) to read.
`ocrType`	No	OCR mode passed to Python.

Field	Required	Notes
`validateType`	No	Validation mode passed to Python.

Response `200`:

```
{
  "ocrJson": "{...raw OCR JSON from Python...}",
  "ocrFieldList": [
    { "name": "Invoice Number", "value": "INV-1001", "type": "SHORT_TEXT" },
    { "name": "Invoice Date",   "value": "2026-10-01", "type": "DATE" }
  ],
  "ocrText": "full document text..."
}
```

1b. `POST /api/uploadAndIndex/uploadWithOcr`

Stages the file in monitor storage using the OCR values you already have. **Does not call OCR again.**

`multipart/form-data`:

Field	Required	Notes
`file`	**Yes**	Same file as 1a.
`repositoryId`	**Yes**	Repository GUID.
`metadata`	No*	Flat JSON object of final values, e.g. `{"Invoice Number":"INV-1001","Invoice Date":"2026-10-01"}`. Preferred.
`fields`	No*	Alternative to `metadata`: the `ocrFieldList` array from 1a. Used only when `metadata` is empty.
`ocrJson`	No	`ocrJson` from 1a (kept with the stage row).
`ocrText`	No	`ocrText` from 1a.
`filename`	No	Override the file name. Defaults to the uploaded file's name.

* Send `metadata` or `fields` (one of them).

Response `200`:

```
{
  "fileId": "7d1c...-stage-guid",
  "repositoryId": "0b9d...",
  "fileName": "invoice.pdf",
  "filePath": "monitor/0b9d.../20261027103000/invoice.pdf",
  "ocrJson": "{...}",
  "ocrFieldList": [ { "name": "Invoice Number", "value": "INV-1001", "type": null } ]
}
```

View the staged file: `GET /api/uploadAndIndex/files/{fileId}` or `GET /api/uploadAndIndex/files/by-path?path=<filePath>` (`&base64=true` returns JSON with base64).

1c. `PUT /api/uploadAndIndex/index/{fileId}` — save & archive

Saves field values and moves the staged file into the repository archive (synchronous).

```
{
  "repositoryId": "0b9d...",
  "itemId": null,
  "status": "Indexed",
  "fields": [ { "name": "Invoice Number", "value": "INV-1001" } ],
  "ocrResult": null
}
```

Response: `{ "stageId", "hangfireJobId", "message", "itemId", "fileName", "filePath" }`. `404` if the stage row isn't found.

1d. `PUT /api/uploadAndIndex/index/{fileId}/fields`

Updates the staged field values only (no archive, status unchanged). Body: `{ "fields": [ { "name": "...", "value": "..." } ], "repositoryId": "..." }`.

1e. `POST /api/uploadAndIndex/index/deleteFiles`

Soft-deletes staged files and removes their temp blobs. Body: `{ "fileIds": ["guid", "..."], "repositoryId": "optional" }` (alias `ids`, or single `fileId`). Response: `{ "requested", "succeeded", "failed", "files": [ { "fileId", "succeeded", "error" } ] }`.

Other related endpoints: `POST /api/uploadAndIndex/upload` (stage without OCR), `POST /api/uploadAndIndex/bulkUpload` (many files, background OCR, returns `jobId`), `POST /api/uploadAndIndex/index/all` (list staged files).

---

2. Forms (designer)

2a. `POST /api/form` — create

Body (JSON) is the form designer payload. Either send it directly (must contain `settings` and/or `panels`) or wrap it in `formJson`:

```
{
  "formJson": {
    "settings": {
      "general": { "name": "Vendor Master", "description": "", "type": "MASTER", "uniqueColumns": [] },
      "publish": { "publishOption": "PUBLISHED" }
    },
    "panels": [ { "id": "p1", "fields": [ { "id": "f1", "label": "Vendor Name", "type": "SHORT_TEXT" } ] } ],
    "secondaryPanels": []
  }
}
```

Status	Meaning	Body
`201`	Created	the new `formId` (string); `Location: /api/form/{formId}`
`406`	Form name already exists	message
`404`	Could not create	message
`400`	Body isn't a designer JSON	`{ "error": ... }`

2b. `PUT /api/form/{formId}` — update

Same body as create. `200` → `formId`; `406` name conflict; `404` not found.

2c. `DELETE /api/form/{formId}` — delete

Soft-deletes the form. `204` on success, `404` if not found.

2d. `GET /api/form/{formId}`

Returns `{ id, name, description, type, layout, publishOption, createdBy, modifiedBy, createdAt, modifiedAt, formJson }`.

2e. List forms

- `GET /api/form/all` → `{ "items": [ { formId, formName, createdBy, modifiedBy, createdByName, modifiedByName } ] }`
- `POST /api/form/all` → paged/filtered, same shape as workflow list:

```
{
  "sortBy": { "criteria": "modifiedAt", "order": "DESC" },
  "groupBy": "",
  "filterBy": [ { "groupCondition": "AND", "filters": [ { "criteria": "name", "condition": "contains", "value": "vendor" } ] } ],
  "currentPage": 1,
  "itemsPerPage": 20,
  "mode": "browse"
}
```

Response: `{ "data": [ { "key": "", "value": [ ...forms ] } ], "meta": { currentPage, itemsPerPage, totalItems } }`.

---

3. Form entries

Field keys in an entry body can be the control **label**, its **jsonId**, or the physical **column name**. Use `GET /api/form/{formId}/entry/controls` to get the field list.

3a / 3b. Create or update — `POST|PUT /api/form/{formId}/entry/{entryId}`

- **Create:** `entryId = 00000000-0000-0000-0000-000000000000`
- **Update:** `entryId` = existing entry (item) GUID

```
{ "fields": { "Vendor Name": "Acme Supplies", "Vendor Code": "3101", "Email": "ap@acme.com" } }
```

A flat object (without `fields`) is also accepted.

Response body `{ "id", "output", "EncryptOutput", "creditconsumed" }`. The status code tells you the result:

Status	id	Meaning
`201`	1	Entry created (`output` = new entry id)
`202`	2	Entry updated
`409`	3	Duplicate value in a unique column (`output` lists the fields)
`404`	0	Form/entry not found

3c. `POST /api/form/{formId}/entry/upsert` — insert or update by column

Update the row that matches, or insert when nothing matches. No match key → always insert.

```
{
  "fields": { "Vendor Name": "Acme Supplies", "Email": "ap@acme.com" },
  "matchColumn": "Vendor Code",
  "matchValue": "3101",
  "matchFilters": [ { "columnName": "Country", "columnValue": "IN" } ]
}
```

Response `200`: `{ status, message, formId, columnName, resolvedColumn, entries: [ savedRow ], updatedCount }`.

3d. `PUT /api/form/{formId}/entry/by-column` — update by column

Updates the **single** entry where `columnName = columnValue`. Returns `400` if zero or more than one entry matches.

```
{ "columnName": "Vendor Code", "columnValue": "3101", "fields": { "Email": "new@acme.com" } }
```

`POST /api/form/{formId}/entry/by-column` reads by column: with `columnValue` → matching `entries`; without it → distinct `values` for that column. Optional `filters` narrow both.

3h. `DELETE /api/form/{formId}/entry/{entryId}` — delete entry

Soft delete: sets `is_deleted = true` (and `modified_at` / `modified_by`). The row is kept, so history and workflow links still work, but get, list, by-column and upsert no longer return or match it. No request body.

Status	Meaning	Body
`200`	Deleted	`{ "status": 1, "entryId": "c3f1...", "message": "deleted form entry" }`
`404`	Form table or entry not found, or already deleted	`{ "error": "form entry not found" }`
`400`	Tenant/user context missing	`{ "error": "..." }`

3e. `GET /api/form/{formId}/entry/{entryId}`

Returns an array with the entry row(s) as key/value objects. `404` if not found.

3f. List entries — `POST /api/form/{formId}/entry/all`

```
{
  "sortBy": { "criteria": "createdAt", "order": "DESC" },
  "filterBy": [ { "groupCondition": "AND", "filters": [ { "criteria": "Vendor Code", "condition": "eq", "value":
"3101" } ] } ],
  "currentPage": 1,
  "itemsPerPage": 20,
  "includeFormJson": true
}
```

Filter `condition`: `contains` (alias `like`), `eq` (`=`), `neq` (`!=`).

Response: `{ status, formId, entries: [ {...row} ], meta: { currentPage, itemsPerPage, totalItems } }`. Shortcut: `GET /api/form/{formId}/entry/all?currentPage=1&itemsPerPage=20&includeFormJson=true`.

3g. `GET /api/form/{formId}/entry/controls`

Field schema: `{ formId, controlCount, controls: [ { id, jsonId, name, type, isMandatory, parentId, columnName, validationJson, ... } ] }`.

---

4. Repository file upload

Both endpoints are `multipart/form-data` (max 100 MB) and need **Upload** permission on the repository.

Field	Required	Notes
`file`	**Yes**	The document.
`metadata`	No	JSON object of repository field values, e.g. `{"Supplier":"Acme","PoNumber":"PO-55","DocumentType":"Invoice"}`. Extra form fields are merged in too.
`workflowId`	No	Link to a workflow (with `processId` and/or `instanceId`).
`processId`	No	Workflow process (ticket) id.
`instanceId`	No	Workflow instance GUID.
`transactionId`	No	Workflow transaction id.
`storageProviderCode`	No	Storage override; defaults to the tenant's storage.

4a. `POST /api/repositories/{repoId}/items/upload-archive` — folder layout (recommended)

Stores the file as `ezts{tenantId}/archive/{repositoryName}/{folder fields...}/{fileName}.ext`. Folder levels come from repository fields marked **IncludeInFolderStructure**, so `metadata` **must** contain values for the mandatory folder levels.

4b. `POST /api/repositories/{repoId}/items/upload` — flat path

Same fields, without the folder layout.

Response `201` (both):

```
{
  "itemId": "c3f1...",
  "fileName": "PO-55.pdf",
  "filePath": "archive/Invoices/Acme/Invoice/PO-55.pdf",
  "storageProviderCode": "EZOFIS",
  "fileVersion": 1,
  "workflowAttached": true,
  "workflowId": "…", "processId": 120, "instanceId": "…"
}
```

4c. `PATCH /api/repositories/{repoId}/items/{itemId}/metadata`

Use this for the **Save** button on the archived-file metadata panel. Send only the fields shown in the panel (keys = field name or column, same as upload metadata):

```
{ "metadata": { "Billing Address": "1800 Woodward Dr, Ottawa, ON, K2C 0P7", "Order Number": "EST-900004" } }
```

The update and its history are saved in one transaction. Only fields whose value **actually changed** are recorded (`1200` vs `1200.00`, or the same text with extra spaces, is not a change).

Response:

```
{
  "itemId": "c3f1...",
  "updatedFieldCount": 2,
  "changeId": "9a7e...",
  "changes": [
    { "field": "Order Number", "column": "OrderNumber", "oldValue": "EST-900003", "newValue": "EST-900004" }
  ],
  "item": { "...": "full item detail after save" }
}
```

`changes` is empty and `changeId` is `null` when nothing changed. Needs **EditMetadata** permission.

4e. Metadata edit history

Every metadata save that changes at least one value is stored in `repository."ItemMetadataHistory"` (one row per field, grouped by `changeId`; the table is created automatically on first save). It is shown two ways:

a) In the Timeline tab — `GET /api/repositories/{repoId}/items/{itemId}/timeline` now includes one event per save:

```
{
  "id": "9a7e...",
  "eventType": "metadata",
  "title": "Metadata edited — Order Number",
  "description": "Order Number: EST-900003 → EST-900004",
  "actorType": "User",
  "actorName": "Jake Gillis",
  "createdAtUtc": "2026-10-27T09:15:02Z",
  "isDerived": false,
  "metadataChanges": [
    { "field": "Order Number", "column": "OrderNumber", "oldValue": "EST-900003", "newValue": "EST-900004" }
  ]
}
```

When several fields change in one save the title is `Metadata edited (N fields)` and `metadataChanges` lists each one. Use `metadataChanges` to render an old → new table under the timeline entry; `description` is a ready-made one-line summary. Empty values show as `(empty)` in `description` and `null` in `metadataChanges`. Other timeline events have `metadataChanges: null`.

b) Dedicated list — `GET /api/repositories/{repoId}/items/{itemId}/metadata/history` (newest first, needs **View** permission):

```
{
  "itemId": "c3f1...",
  "count": 1,
  "entries": [
    {
      "changeId": "9a7e...",
      "changedAtUtc": "2026-10-27T09:15:02Z",
      "changedBy": "2eb6...",
      "changedByName": "Jake Gillis",
      "changes": [
        { "field": "Order Number", "column": "OrderNumber", "oldValue": "EST-900003", "newValue": "EST-900004" }
      ]
    }
  ]
}
```

`404` when the item does not exist. Edits made by the AP agent through the same update path are recorded too (`changedBy` may be `null`, shown as actor type `System`).

4d. `DELETE /api/repositories/{repoId}/items/{itemId}`

Soft delete. Response: `{ "itemId": "...", "isDeleted": true }`.

---

5. Repository structure & items

5a. `GET /api/repositories/{repoId}/browse/structure`

Returns the folder levels and the available drill-down paths:

```
{
  "folderFields": [
    { "level": 1, "name": "Supplier", "sqlColumnName": "Supplier" },
    { "level": 2, "name": "Document Type", "sqlColumnName": "DocumentType" }
  ],
  "browsePaths": [
    { "id": "by-Supplier", "label": "Supplier → Document Type", "fieldOrder": ["Supplier", "DocumentType"] }
  ]
}
```

The last folder level (the file name) is left out. Clicking the level before last lists files.

5b. `GET /api/repositories/{repoId}/browse/children`

Gets the next tree level.

Query	Notes
`pathId`	From `browsePaths[].id`. Omit to use the first path.
`parentFilters`	JSON of the folders already picked, keys = `sqlColumnName`. Level 1: omit or `{}`; level 2: `{"Supplier":"Acme Supplies"}`. URL-encode it.
`page`, `pageSize`	Default `1`, `50`.
`search`	Filter group names.

Response: `{ level, groupField, groupFieldName, pathId, pathLabel, parentFilters, isLeafLevel, groups }`. `groups` is paged; each group is `{ name, itemCount, dateModified }`. Also: `GET /api/repositories/{repoId}/browse/groups/{fieldName}` groups by any folder field.

5c. `GET /api/repositories/{repoId}/items` — file list

Query	Default	Notes
`filters`	—	JSON; keys from `items/filter-fields`. `{"Supplier":"Acme"}`; multi-value `{"Status":["Verifier","Approved"]}`.
`search`	—	Free text.
`dateFrom`, `dateTo`	—	Document date range.
`sortBy` / `sortOrder`	`documentDate` / `desc`	
`page` / `pageSize`	`1` / `50`	`pageSize` 1–500.

Query	Default	Notes
`skipTotal`	`false`	`true` skips the count (`totalCount = -1`), so loads are faster.
`cursor`	—	`nextCursor` from the previous page, for infinite scroll.

To show files inside a folder, pass the folder's `parentFilters` as `filters`.

5d. `POST /api/repositories/{repoId}/items/query` (recommended for multi-select)

Same as 5c, but filters go in the body:

```
{
  "filters": { "Supplier": ["CHAMPION INDUSTRIAL SUPPLY", "Gerrie Logistics Services Ltd"], "Currency": "CAD" },
  "search": null,
  "sortBy": "documentDate",
  "sortOrder": "desc",
  "page": 1,
  "pageSize": 50
}
```

Item row: `{ id, fileName, fileVersion, documentType, supplier, invoiceNumber, poNumber, documentDate, amount, currency, status, DetailsRow: [...], lineItems }`.

5e. `GET /api/repositories/{repoId}/items/filter-fields`

`{ "fields": [ { "name": "Supplier", "sqlColumnName": "Supplier", "dataType": "SHORT_TEXT" } ] }` — the allowed `filters` keys.

5f. Item detail & file

- `GET /api/repositories/{repoId}/items/{itemId}` — full item detail
- `GET /api/repositories/{repoId}/items/{itemId}/file?disposition=inline|attachment` — view or download the file

---

6. Indexed vs archived files

The UI shows two lists. They come from **different endpoints**, and a file moves from the first to the second when it is archived.

	Index list (waiting to be indexed)	Archived files (in the repository)
Endpoint	`POST /api/uploadAndIndex/index/all`	`GET /api/repositories/{repoId}/items` or `POST .../items/query`
Folder view	—	`GET .../browse/structure`, `GET .../browse/children`
Which files	Uploaded by the **logged-in user**, not deleted, not archived	All archived files the user has **View** permission for
Row id	`fileId` (temporary/stage id)	`itemId` (repository item id)
View / download	`GET /api/uploadAndIndex/files/{fileId}` (`?disposition=attachment` to download)	`GET /api/repositories/{repoId}/items/{itemId}/file`
Edit values	`PUT /api/uploadAndIndex/index/{fileId}/fields`	`PATCH .../items/{itemId}/metadata`
Delete	`POST /api/uploadAndIndex/index/deleteFiles`	`DELETE .../items/{itemId}`

6a. `POST /api/uploadAndIndex/index/all` — index list

```
{ "repositoryId": "0b9d...", "currentPage": 1, "itemsPerPage": 50, "mode": "browse" }
```

- `repositoryId` is required (repository GUID).
- `mode: "browse"` (default) — pending files only. `mode: "trash"` removes the filters and also returns **deleted and archived** rows.

Response:

```
{
  "data": [ { "key": "", "value": [
    {
      "id": "7d1c...", "fileId": "7d1c...", "name": "invoice.pdf",
      "status": "OCR", "stageStatus": "OCR",
      "repositoryId": "0b9d...", "repositoryName": "Invoices",
      "size": 182044, "fileType": "pdf", "createdAt": "2026-10-27T05:00:00.0000000Z",
      "filePath": "monitor/0b9d.../20261027103000/invoice.pdf",
      "fields": [ { "name": "Invoice Number", "value": "INV-1001" } ],
      "fileUrl": "/api/uploadAndIndex/files/7d1c...",
      "downloadUrl": "/api/uploadAndIndex/files/7d1c...?disposition=attachment",
      "itemId": null, "promotedItemId": null,
      "itemFileUrl": null, "itemDownloadUrl": null,
      "isIndexed": false
    }
  ] } ],
  "meta": { "currentPage": 1, "itemsPerPage": 50, "totalItems": 1 }
}
```

> **Note:** `isIndexed`, `itemId` and `itemFileUrl` are only filled in after a file is archived. Archived files are left out in `browse` mode, so there `isIndexed` is always `false`. Use the repository `items` endpoints to show archived files.

6b. From index list to archived

1. Upload: `uploadWithOcr`, `upload` or `bulkUpload` → the file appears in `index/all`.
2. Review/edit: `PUT /api/uploadAndIndex/index/{fileId}/fields` (optional).
3. Archive: `PUT /api/uploadAndIndex/index/{fileId}` → the file is copied into the repository and the response returns `itemId`.
4. The stage row is marked archived (`status = ARCHIVED`, linked to `itemId`), so it **leaves `index/all`** and **shows in repository `items`**.

6c. Bulk upload progress — `GET /api/uploadAndIndex/bulkUpload/jobs/{jobId}`

After `POST /api/uploadAndIndex/bulkUpload`, poll this to show each file's phase:

Phase	Meaning
`PendingOcr`	Uploaded, OCR not finished
`OcrCompleted`	OCR done, ready to index
`OcrFailed`	OCR failed
`Indexed`	Archived into the repository

`GET /api/uploadAndIndex/bulkUpload/jobs/active` returns the user's running job, if any.

Errors (all endpoints)

Status	When
`400`	Missing file / `repositoryId`, invalid JSON, validation failure → `{ "error": "..." }`
`401`	Missing or invalid token
`403`	No repository permission (View / Upload / Delete / EditMetadata)
`404`	Form, entry, repository, item or stage file not found
`406`	Form name conflict (create/update form)
`409`	Duplicate unique value (form entry)